NumberForge — User Guide

Version 1.0 For questions or issues: nregoblin@gmail.com

Contents

1. Overview
2. Installation
3. Quick Start
4. Creating BigNumbers
5. Arithmetic
6. Comparison
7. Math Utilities
8. Formatting
9. Format Options
10. Custom Suffix Lists
11. Changing the Global Formatter
12. Parsing and Serialization
13. Demo Scene
14. Support

---

1. Overview

NumberForge provides a `BigNumber` struct for representing arbitrarily large positive or negative numbers. It is designed for idle games, incremental games, RPGs, and any project where values routinely exceed what a `double` or `long` can hold.

Values are stored as a normalized mantissa/exponent pair — `value = Mantissa × 10^Exponent` — giving around 15 significant digits of precision at any scale, from everyday pocket-change values up to numbers with hundreds of digits.

---

2. Installation

Import the package from the Unity Asset Store. NumberForge requires no additional dependencies and works with all render pipelines. Unity 2022.3 LTS or later is recommended.

After import, the assembly `ScriptGoblin.NumberForge` is available to all scripts in your

project automatically.

---

3. Quick Start

```
using ScriptGoblin.NumberForge;
using ScriptGoblin.NumberForge.Formatting;

// Create a big number
var gold = new BigNumber(1, 6); // 1,000,000

// Arithmetic works naturally
gold += 500000;
gold *= 2;

// Display it
Debug.Log(gold.ToFormattedString()); // 3M
```

---

4. Creating BigNumbers

From mantissa and exponent

The most direct way. The mantissa does not need to be pre-normalized.

```
var a = new BigNumber(1.5, 6);   // 1,500,000
var b = new BigNumber(750, 3);   // 750,000 (normalized automatically)
var c = new BigNumber(1, 100);   // 1e100
```

From primitive types

`int`, `long`, and `double` convert implicitly.

```
BigNumber x = 42;
BigNumber y = 1_000_000L;
BigNumber z = 3.14;
```

From a string

```
var a = new BigNumber("1500000");   // plain integer
var b = new BigNumber("1.5e6");     // scientific notation
var c = new BigNumber("1,500,000"); // commas are ignored
```

Named constants

```
BigNumber.Zero    // 0
BigNumber.One     // 1
BigNumber.Epsilon // a very small positive value, useful as a near-zero
threshold
```

---

5. Arithmetic

All four standard operators are supported between two `BigNumber` values, and between a

`BigNumber` and an `int`, `long`, or `double`.

```
var a = new BigNumber(1, 9); // 1 billion
var b = new BigNumber(5, 8); // 500 million

var sum        = a + b;      // 1.5 billion
var difference = a - b;      // 500 million
var product    = a * b;      // 5e17
var quotient   = a / b;      // 2

// Mixed types — no manual casting needed
var doubled   = a * 2;
var increased = a + 250_000L;
var scaled    = a * 0.5;
```

Division by zero throws `DivideByZeroException`.

Unary negation

```
var negative = -a; // -1 billion
```

Precision note

When two values differ by more than 15 orders of magnitude, the smaller one has no effect on the result. For example, adding `1e100` to `1` returns `1e100` unchanged. This matches standard floating-point behavior.

---

6. Comparison

All comparison operators are available, plus `IComparable<BigNumber>` and `IEquatable<BigNumber>` for use with sorting and collections.

```
var a = new BigNumber(1, 9);
var b = new BigNumber(5, 8);

bool greater  = a > b;   // true
bool equal    = a == b;  // false

// Convenience properties
bool isZero = a.IsZero;
bool isPos  = a.IsPositive;
bool isNeg  = a.IsNegative;
```

---

7. Math Utilities

Abs, Min, Max, Clamp

```
var a = new BigNumber(-5, 3);
var b = new BigNumber( 2, 3);

BigNumber abs     = a.Abs();                                  // 5,000
BigNumber min     = BigNumber.Min(a, b);                     // -5,000
BigNumber max     = BigNumber.Max(a, b);                     // 2,000
BigNumber clamped = BigNumber.Clamp(value, min, max);
```

Floor and Ceil

Returns the nearest integer value downward or upward. Values with an exponent of 15 or more are already effectively integers and are returned unchanged.

```
var x = new BigNumber(1.7, 0); // 1.7

BigNumber floored = x.Floor(); // 1
BigNumber ceiled  = x.Ceil();  // 2
```

Lerp

Linearly interpolates between two `BigNumber` values. The `t` parameter is clamped to [0, 1].

```
var from = new BigNumber(1, 0);  // 1
var to   = new BigNumber(1, 6);  // 1,000,000

BigNumber halfway = BigNumber.Lerp(from, to, 0.5); // ~500,000

// Typical use: animate a UI counter toward a target
_display = BigNumber.Lerp(_display, _target, Time.deltaTime * 5f);
```

Pow

Raises the value to a power. Supports fractional and negative exponents.

```
var base  = new BigNumber(2, 0); // 2

BigNumber squared = base.Pow(2);    // 4
BigNumber sqrt    = base.Pow(0.5);  // ~1.414
BigNumber inverse = base.Pow(-1);   // 0.5
```

Sqrt

Shorthand for the square root.

```
var x    = new BigNumber(9, 0);
var root = x.Sqrt(); // 3
```

Throws `InvalidOperationException` for negative values.

Log10

Returns the base-10 logarithm as a plain `double`. Useful for scaling UI elements or progress bars.

```
var gold  = new BigNumber(1, 6); // 1,000,000
double log = gold.Log10();        // 6.0

// Scale a progress bar: normalize against a maximum exponent
float fill = (float)(gold.Log10() / maxExponent);
```

Throws `InvalidOperationException` for zero or negative values.

ToDouble and ToLong

Convert back to a primitive for use with APIs that do not accept `BigNumber`. Overflow returns `Infinity` for `double` and throws `OverflowException` for `long`.

```
var small = new BigNumber(1.5, 3); // 1,500

double d = small.ToDouble(); // 1500.0
long   l = small.ToLong();   // 1500
```

---

8. Formatting

`BigNumber` separates representation from display. `ToString()` always returns the raw normalized form suitable for logging and save data. To get a display-ready string, use `ToFormattedString()` or pass a `FormatOptions` instance.

Four built-in notations

Notation	**Example output for 1,500,000**
Short	1.5M
Long	1.5 Million
Scientific	1.5e6
Engineering	1.5e6

Engineering differs from Scientific when the value is not an exact power of three: `15,000` formats as `15e3` in Engineering but `1.5e4` in Scientific.

Quick formatting

```
// Uses BigNumberFormatter.Current (Short by default)
string display = gold.ToFormattedString();

// Supply options inline
var opts = new FormatOptions { notation = NumberNotation.Long };
string display = gold.ToFormattedString(opts);
```

IFormattable shorthand

`BigNumber` implements `IFormattable`, so format strings work in interpolation.

```
string s = $"{gold:S}"; // Short
string s = $"{gold:L}"; // Long
string s = $"{gold:E}"; // Scientific
string s = $"{gold:G}"; // Engineering
```

---

9. Format Options

`FormatOptions` lets you control every aspect of the output per call without touching any global state.

```
var opts = new FormatOptions
{
    notation          = NumberNotation.Short,
```

```
    decimals          = 3,       // up to 3 decimal places
    trimTrailingZeros = true,    // "1.50M" becomes "1.5M"
    positiveSign      = "+",     // prepend "+" for positive values
    negativeSign      = "-"      // default
};

string result = gold.ToFormattedString(opts);
```

decimals

Controls the maximum number of decimal places shown. Default is `2`.

```
// 1,500,000 with different decimal settings
new FormatOptions { decimals = 0 } // 2M
new FormatOptions { decimals = 2 } // 1.5M
new FormatOptions { decimals = 4 } // 1.5M (trailing zeros trimmed by default)
new FormatOptions { decimals = 4, trimTrailingZeros = false } // 1.5000M
```

positiveSign

Useful for showing signed differences, such as damage numbers or stat changes.

```
var opts = new FormatOptions { positiveSign = "+" };

var gain = new BigNumber(5, 3);
var loss = new BigNumber(-2, 3);

gold.ToFormattedString(opts); // "+5K"
loss.ToFormattedString(opts); // "-2K"
```

---

10. Custom Suffix Lists

By default, Short format uses K / M / B / T and Long format uses Thousand / Million / Billion. You can replace these lists project-wide by subclassing `BigNumberNames` and assigning your instance to `BigNumberNames.Current`.

```
public class IdleGameNames : BigNumberNames
{
    public override string[] ShortNames => new[]
        { "k", "m", "b", "t", "aa", "ab", "ac", "ad" };
}

// Call once at startup, before any formatting
BigNumberNames.Current = new IdleGameNames();
```

After this, `new BigNumber(1, 3).ToFormattedString()` returns `"1k"` instead of `"1K"`.

Values that exceed the length of your custom list fall back to scientific notation automatically.

---

11. Changing the Global Formatter

`BigNumberFormatter.Current` controls what `ToFormattedString()` and

`BigNumber.ToString()` use when no options are passed. Replace it once at startup.

```
// Switch the default to Scientific
BigNumberFormatter.Current = new ScientificFormat();

// From this point, gold.ToFormattedString() uses scientific notation
```

The available formatter classes are `ShortFormat`, `LongFormat`, `ScientificFormat`, and `EngineeringFormat`. You can also implement `IBigNumberFormatter` to create a completely custom formatter.

```
public class MyFormatter : IBigNumberFormatter
{
    public string Format(BigNumber value, FormatOptions options = null)
    {
        // your custom logic
        return $"{value.Mantissa:F2} × 10^{value.Exponent}";
    }
}

BigNumberFormatter.Current = new MyFormatter();
```

---

12. Parsing and Serialization

ToString and TryParse

`BigNumber.ToString()` returns the raw form `mantissaeExponent`, for example `1.5e6`. This form is safe for save data and string interpolation and round-trips perfectly through `TryParse`.

```
// Save
string saved = gold.ToString(); // "1.5e6"
PlayerPrefs.SetString("gold", saved);

// Load
string raw = PlayerPrefs.GetString("gold");
if (BigNumber.TryParse(raw, out BigNumber loaded))
    gold = loaded;
```

Supported parse formats

```
new BigNumber("1500000")   // plain integer
new BigNumber("1,500,000") // commas ignored
new BigNumber("1.5e6")     // scientific notation
new BigNumber("1.5")       // decimal
```

Parsing throws `FormatException` with a clear message if the input is not recognized.

Unity serialization

Unity's built-in serializer does not support `double` or `long` fields on structs, so a `BigNumber` field on a `MonoBehaviour` will not appear in the Inspector. The recommended approach is to store the value as a `string` and convert in `Awake`.

```
[SerializeField] private string _goldSave = "0";
```

```
private BigNumber _gold;

private void Awake()
{
    BigNumber.TryParse(_goldSave, out _gold);
}

private void OnApplicationQuit()
{
    _goldSave = _gold.ToString();
}
```

---

13. Demo Scene

A demo scene is included at `Assets/ScriptGoblin/NumberForge/Demo/NumberForgeCalculatorDemo.unity`. Open it and enter Play mode to explore a working calculator that showcases:

- Addition, subtraction, multiplication, and division between BigNumber values
- Preset buttons for common large values (1K, 1M, 1B, 1T, 1e50, 1e100)
- Live format switching between Short, Long, Scientific, and Engineering
- Lerp with an adjustable slider and live output in all four formats
- Abs, Sqrt, Pow², and Log10 of the current value
- FormatOptions examples showing decimal control and positive sign prefix

A second animated showcase scene is included at `Assets/ScriptGoblin/NumberForge/Demo/NumberForgePromoShot.unity` and demonstrates BigNumber in a simulated idle-game HUD.

---

14. Support

If you run into any issues, have a feature request, or just want to say hello — reach out at **nregoblin@gmail.com**. Quick replies guaranteed.